

Form Submission Data -> Apps Visualization System

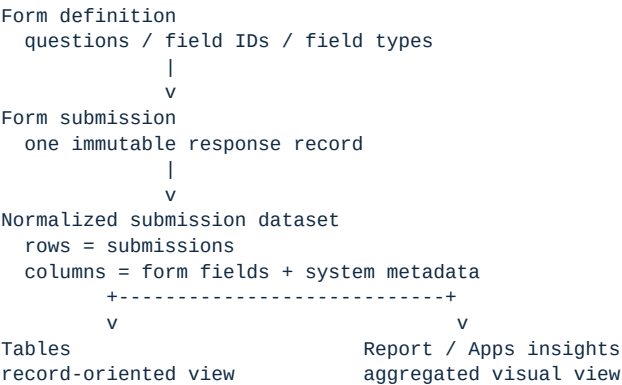
Product and architecture specification

Purpose: Give an AI or product engineer an unambiguous model for bringing the Jotform Report pattern into Jotform Apps, then extending it safely.

Status: Product/architecture specification. It describes the intended Apps implementation, informed by a live Jotform Form -> Tables -> Report test on 15 August 2026. It is not a claim that Apps already has a backend submission integration.

1. The core idea

A form is a schema and each completed form is a submission record. Tables and Reports are two views over the same submission dataset:



The report is not a separate manually maintained dataset. A chart is a saved query/configuration over a source dataset. When a new submission enters the source, every chart that includes it should recompute from the same data.

Live example verified

The Hotel Booking Form test used this form submission:

Field	Test value
Name	Report Flow Test
E-mail	report-flow-test-20260815@example.com
Room Type	Family Room (1 to 4 People)
Number of Guests	3

Field	Test value
Arrival Date & Time	20 Aug 2026, 1:30 PM
Departure Date	23 Aug 2026
Free Pickup?	Yes
Special Requests	REPORT-FLOW-TEST-20260815

After submit, it appeared as a new row in Jotform Tables. The table had one Submission Date system column and one column for each form field. The same table contains a Report tab, confirming the report is attached to that data source rather than receiving hand-entered chart values.

2. The canonical data model for Apps

Keep data at the app level, not inside a chart element's properties. This follows the existing Apps architectural direction: elements bind to central appData; they must not own duplicate copies of business data.

```
type FieldKind =
  | 'shortText'
  | 'longText'
  | 'email'
  | 'number'
  | 'singleChoice'
  | 'multipleChoice'
  | 'boolean'
  | 'date'
  | 'dateTime'
  | 'rating'
  | 'file'
  | 'address'
  | 'unknown';

interface FormFieldDefinition {
  id: string;           // Stable provider/Jotform field ID, never the display label
  label: string;        // "Room Type"
  kind: FieldKind;
  options?: Array<{ id: string; label: string }>;
  isSensitive?: boolean; // E-mail, phone, address and other PII
}

interface SubmissionRecord {
  id: string;           // Stable submission ID
  formId: string;
  submittedAt: string;  // ISO date-time
  updatedAt?: string;
  status?: 'submitted' | 'partial' | 'deleted';
  answers: Record<string, unknown>; // keyed by FormFieldDefinition.id
}

interface FormSubmissionSource {
  id: string;           // e.g. `jotform-form:<formId>`
  provider: 'jotform';
  formId: string;
  title: string;
  fields: FormFieldDefinition[];
  submissions: SubmissionRecord[];
  lastSyncedAt?: string;
}

interface AppData {
  formSources?: FormSubmissionSource[];
  // existing app-level entities such as products, orders and coupons stay here too
}
```

Non-negotiable identity rules

1. Bind by `formId`, `field.id`, and `submission.id`-never by the visible field label.
 2. A label can be renamed without breaking a chart.
 3. Preserve a field's kind in the snapshot. It drives valid aggregations and controls.
 4. Store raw answer values. Do not persist chart-specific totals as the source of truth.
 5. Keep system fields (`submittedAt`, `status`, `source/form identity`) separate from answers, while allowing them to be used as report dimensions/filters.
-

3. What a chart actually is

A chart is a serializable configuration that evaluates a dataset. It is not a set of hardcoded data points.

```
type ChartKind =
  | 'bar'
  | 'horizontalBar'
  | 'line'
  | 'area'
  | 'pie'
  | 'donut'
  | 'metric'
  | 'table'
  | 'list';

type Aggregation = 'count' | 'sum' | 'average' | 'minimum' | 'maximum' | 'none';

interface FilterRule {
  fieldId: string;
  operator:
    | 'equals'
    | 'notEquals'
    | 'includes'
    | 'isEmpty'
    | 'isNotEmpty'
    | 'greaterThan'
    | 'lessThan'
    | 'between'
    | 'before'
    | 'after';
  value?: unknown;
}

interface ChartConfig {
  id: string;
  sourceId: string;
  kind: ChartKind;
  title?: string;
  dimension?: { fieldId: string; dateGranularity?: 'day' | 'week' | 'month' | 'year' };
  measure: { aggregation: Aggregation; fieldId?: string };
  filters: FilterRule[];
  sort?: { by: 'dimension' | 'measure'; direction: 'asc' | 'desc' };
  limit?: number;
  emptyState?: { title: string; description?: string };
}
```

`ChartConfig` belongs to the visualization element/page layout. Its input data belongs to `appData.formSources`. The chart always resolves its input from `sourceId` at render time.

4. Evaluation pipeline

This is the exact conceptual pipeline the runtime must follow:

1. Resolve `sourceId`
2. Select active submission records

3. Apply global/page filters
4. Apply this chart's filters
5. Extract dimension and measure values by stable field ID
6. Normalize values by field type
7. Group by dimension
8. Aggregate each group
9. Sort and limit the result
10. Render the chosen visualization or its empty/error state

Pseudocode:

```
function evaluateChart(chart: ChartConfig, appData: AppData) {
  const source = appData.formSources?.find((item) => item.id === chart.sourceId);
  if (!source) return { state: 'missing-source' as const };

  const records = source.submissions
    .filter((item) => item.status !== 'deleted')
    .filter((item) => matchesFilters(item, chart.filters));

  const rows = records.flatMap((item) => normalizeForChart(item, source.fields, chart));
  const groups = groupBy(rows, (row) => row.dimensionKey);

  return Object.entries(groups).map(([dimensionKey, items]) => ({
    dimensionKey,
    value: aggregate(items, chart.measure),
  }));
}
```

Important normalization rules:

- Multiple-choice answers: explode an answer into one contribution per selected option for distribution charts. Do not turn ['A', 'B'] into a single "A, B" category.
- Dates/date-times: parse once into a normalized date value, then group at the selected day/week/month/year granularity. Respect the app/source timezone consistently.
- Numbers: reject non-numeric/empty answers from sum, average, minimum, and maximum; count may count records or non-empty values depending on the explicit measure setting.
- Choice options: retain an option with zero responses when the form definition declares it and the chart is showing a bounded choice distribution. This prevents a chart from implying that the option does not exist.
- Text/PII: never create a default public chart from e-mail, phone, address, name, or free text. Use restricted table/list elements with an explicit privacy warning instead.

5. Field type -> valid visualization logic

The AI must propose only valid defaults. It can offer alternatives, but must not generate an invalid aggregation such as average of a Room Type.

Field type	Best default	Valid measures	Useful alternatives	Guardrails
Single choice / boolean / rating	Bar or donut	count of responses	horizontal bar, metric for one selected option	Include zero-valued declared options where appropriate
Multiple choice	Horizontal bar	count of selections	bar, table	A response can count in more than one category
Number	Metric or bar by category	sum, average, min, max, count	histogram, line with date dimension	Define whether count means rows or non-empty numeric values
Date / date-time	Line / area	count of submissions; sum/average of a number	bar by month, metric for period	Require a date granularity and timezone
Short / long text	Table/list	none; count non-empty	word/keyword feature only with a dedicated safe implementation	Do not infer a categorical chart from arbitrary text

Field type	Best default	Valid measures	Useful alternatives	Guardrails
E-mail/name/phone/addresses	Restricted table/list	count only, if privacy allows	none by default	Do not expose raw values in public/published Apps
File	Table/list	count of uploads	none	Render only authorized references, never raw file data

Examples from Hotel Booking

- "How many bookings use each room type?" -> dimension: Room Type, measure: count, kind: bar.
- "Average party size by room type" -> dimension: Room Type, measure: average(Number of Guests), kind: bar.
- "Bookings received each month" -> dimension: Submission Date (month), measure: count, kind: line.
- "How many guests requested pickup?" -> dimension: Free Pickup?, measure: count, kind: donut.
- "Upcoming arrivals" -> a filtered table/list, Arrival Date & Time >= now, sorted ascending; this is not best represented by a chart.

6. Apps product experience

Builder flow

1. Owner adds an Insights / Chart element to a page.
2. The element asks for a Form submission source. If no source is connected, show a clear connection/empty state-not fake sample data.
3. The owner selects either a recommended question or a custom chart setup.
4. The builder constrains controls to valid combinations from the field-type table above.
5. The chart previews against the same source data that will be used at runtime.
6. Saving stores only ChartConfig; it does not copy submissions into the element.

Runtime behavior

- The runtime renders a chart from appData . formSources plus its saved configuration.
- New, changed, or deleted submissions trigger source refresh/invalidation and recomputation of affected charts.
- A chart shows an intentional empty state when the current filter returns no records.
- A missing field or disconnected source shows a recoverable configuration state to the owner; visitors should see a neutral unavailable/empty state, not an implementation error.

Builder UI rules for this monorepo

- Builder chrome uses the design-system tokens (--ds-*) and Circular font.
- The canvas/runtime chart element uses app-elements tokens (--fg-*, --bg-*, --space-*, --radius-*).
- Never hardcode visual values or override element styling from the builder. Fix the shared component/style source.
- The element needs semantic states: loading, no source, no data, insufficient configuration, access denied, and error.
- Accessible data alternatives are required: chart title, readable summary, keyboard-accessible legend/tooltip behavior, and an optional tabular view.

7. Filters, scope, and permissions

Filters are queries, not data mutations. They must be composable and inspectable.

```
source records
-> app/global filters
-> page filters
-> element/chart filters
-> aggregation
-> visual result
```

Recommended first-class filters:

- Relative time: today, last 7/30 days, current month, previous month.
- Date range.
- Choice value equality (e.g. Room Type = Family Room).
- Numeric range.
- Presence/absence of an answer.
- Current signed-in user/role, only after the Apps identity model can safely support it.

Permissions are evaluated before aggregation. A visitor must never receive data in a hidden chart and merely have it concealed through CSS. Public published Apps must default to aggregate, non-PII outputs.

8. Feature extensions beyond Jotform Report

These are valuable Apps-specific additions. Build them on the same `ChartConfig` -> `evaluation` -> `renderer` contract.

A. Interactive dashboard filters

Allow one filter element (date range, room type, owner, etc.) to drive multiple chart elements through a page-level filter context. A chart may opt in/out of each filter.

B. Drill-down to records

Clicking a bar, legend item, or metric opens a filtered, permission-aware list of contributing submissions. Preserve the selected dimension as a filter; do not make the user manually recreate it.

C. Goal and comparison metrics

Extend a metric card with a target, previous-period comparison, percent change, and direction. This remains derived data; the target is configuration, not a fake submission.

D. Saved views

Let authorized app users save named filter combinations such as "This month" or "Family rooms, August". Saved views store filter configuration only.

E. Alerts / automation hooks

After a source refresh, evaluate threshold rules such as `daily booking count > 50`. Emit an internal app event only when the state crosses the threshold; avoid sending repeated alerts on every refresh.

F. AI chart assistant

The AI can convert a request into a draft `ChartConfig`, then show its reasoning in plain language before saving:

"I'll group submissions by Room Type and count bookings for the last 30 days."

It must never invent field IDs, data values, source availability, or a chart that is incompatible with the field kind.

9. AI implementation contract

When an AI is asked to add/configure a visualization, it must follow this sequence:

1. Read the available form sources and field definitions.
2. Identify the requested intent: distribution, trend, comparison, total, or record list.
3. Resolve references to stable IDs, while using labels only for human-facing copy.
4. Validate the requested dimension, measure, aggregation, filter values, and privacy level.
5. Produce a draft `ChartConfig` and a one-sentence explanation.
6. Render or preview from the existing data; do not fabricate submissions.
7. If required data is absent, state precisely what is missing and offer the next safe configuration step.

The AI must ask for clarification rather than guess when:

- more than one form source or same-named field could match;
- a measure can mean more than one thing (e.g. `count of submissions` vs `count of filled values`);
- requested output would expose PII or lacks an applicable role/access rule;
- a date-based request lacks a timeframe or timezone that materially affects the outcome.

Required AI output shape

```
interface ChartProposal {
  explanation: string;
  config: ChartConfig;
  validation: {
    sourceFound: boolean;
    fieldsFound: string[];
    warnings: string[];
  };
}
```

10. Acceptance criteria

An implementation is correct when all of the following are true:

- Submitting a form record adds or updates one record in the app-level source.
 - The record's values appear under the matching stable field IDs and flow to a record/table view.
 - A chart configured from that source includes the new record after refresh without editing the chart.
 - Changing a field label does not break existing chart bindings.
 - A deleted/removed field produces a recoverable missing-field state rather than a silently incorrect chart.
 - Multi-select answers are counted per selected option, not serialized into artificial categories.
 - Date aggregation uses one explicit timezone and granularity.
 - Chart configuration never stores a duplicated submission dataset.
 - Public-facing visualizations do not disclose raw PII by default.
 - The same filter definition produces the same result in the chart and its drill-down record view.
 - All UI states are token-based and accessible.
-

11. Recommended delivery order

1. Data contract: FormSubmissionSource, field registry, sync/invalidation behavior.
2. Read-only record view: prove the submission -> app data -> table path.
3. Chart evaluator: filtering, normalization, grouping, aggregation, deterministic tests.
4. One chart element: categorical count bar chart plus empty/loading/error states.
5. Number and date support: metric, average/sum, and time-series trends.
6. Global filters + drill-down.
7. Privacy/access gates, saved views, and AI assistant.

This order keeps the data model reliable before visual complexity is introduced.